



Cómo vamos

Respuesta al análisis "SPACES OS — Estado y alineación con el negocio"

Validación y corrección del análisis previo (hecho solo sobre el manual del demo) con la evidencia real del repositorio auditado.

Proyecto: spaces_doohmain_nueva (demo) · Base: auditoría del repo + análisis de Ana

Verdict: el demo validó producto y flujos; el backend de producción es el track pendiente.

1. Contexto: qué estamos respondiendo

El documento "SPACES OS — Análisis de estado" (preparado por Ana para Jochelo) evalúa el sistema basándose **únicamente en el manual del demo (v1.1)** que generamos. La propia autora aclara que no tuvo acceso al repositorio y marca muchas afirmaciones como `[C] verificar` o `[?] no determinable`.

Como **nosotros sí auditamos el repositorio**, este documento: (a) confirma lo que el análisis acierta, (b) corrige lo que infirió de más o de menos, y (c) responde sus preguntas abiertas con la evidencia real.

2. Veredicto: cómo vamos

Coincidimos en lo esencial: el demo no es evidencia de que el backend de producción esté hecho. Son dos tracks distintos y ahora arranca el que falta.

Alcance	Compleitud	Lectura
Demo (producto, UX, flujos)	~85%	Operación de punta a punta demostrada; producto de-risкеado.
Producción (backend real + comercial-fiscal + integraciones + seguridad)	~20–25%	Es el grueso de lo que falta construir.

3. En qué acierta el análisis (confirmado con el repo)

- **El dato de las pantallas está mockeado.** El adaptador `mock` es el de por defecto y el `http` es un stub. ✓
- **Margen 100% / costo 0.** Es por la seed (no hay rentas cargadas). El motor de margen existe en `derive.ts`, pero necesita datos reales de costo. ✓
- **Hueco comercial-fiscal.** No existen propuestas, método del divisor, presupuestos, aprobación sitio-por-sitio, ODC como módulo, RFC/razón social, pruebas de color, testigos, reportes ni notificaciones. ✓
- **Datos mexicanos bajo marca "Billboards Perú SA"** (CDMX). ✓ Seed mezclada.

4. Lo que el análisis infirió mal (subestima lo que ya hay)

Por trabajar solo con el manual, el análisis **subvalora el backend existente del demo**. Corrección con el repo:

Afirmación del análisis	Realidad en el repositorio
"Auth es mock (password único)"	Auth real: bcrypt, sesiones en tabla <code>sesiones</code> , cookie httpOnly <code>spaces_sesion</code> , y RBAC server-side (<code>rol_permisos</code> + guard <code>exigir(modulo, accion)</code> en cada ruta). El <code>spaces123</code> es solo la contraseña de la seed.
"Backend no evidenciado"	Sí hay backend en el demo: Postgres propio (puerto 5433) con esquema real (<code>db/schema.sql</code> : índices, FKs, triggers) y un BFF (rutas <code>app/api/**</code>) con CRUD real de auth, usuarios, config, sitios, campañas, OT, impresión y cobranzas. Lo híbrido es que las pantallas leen del store mock , no del BFF.
"Módulo Network sin documentar"	Ya está mapeado en nuestra auditoría: toggle programático / tradicional + CMS por pantalla (Broadsign/Invidis/Doohmain) y KPIs de red.
"¿Demo sobre backend real o standalone?" (pregunta abierta)	Standalone con su propio Postgres + BFF. NO corre sobre el Fastify de <code>apps/api</code> (ese existe, pero no está conectado al frontend del demo).

5. Huecos reales confirmados (lo que falta de verdad)

Estos sí son ciertos y son el corazón de lo que falta para producción (cadena comercial-fiscal de SET):

- Propuestas + método del divisor + presupuestos (neto vs. aprobado).
- Aprobación incremental sitio-por-sitio y **ODC** como módulo (hoy solo es una etapa del pipeline).
- RFC / razón social / datos fiscales y **Finanzas real** (folios, IVA, cobranza viva).
- Pruebas de color, testigos y reportes (contratado vs. entregado).
- Notificaciones por evento (vencimientos, aprobaciones, cierres).
- Motor de costos real (renta + impresión + operación) para que el margen deje de ser 100%.
- Conectar el frontend al backend real (implementar el adaptador `http` detrás de `client.ts`).

6. Respuesta a lo que pide el análisis (su \$13)

El análisis pide 6 insumos para pasar de "estimación sobre el manual" a auditoría técnica. Estado:

Lo que pide	Qué ya tenemos
Esquema de datos (entidades, relaciones, enums)	Disponible — <code>db/schema.sql</code> en el repo + sección 3 de nuestra auditoría.
Lista de endpoints / rutas	Disponible — rutas <code>app/api/**</code> inventariadas en la auditoría (sección 6.3).
Estado del adaptador <code>http</code> en <code>lib/data/</code>	Aclarado — <code>mock</code> por defecto; <code>http</code> es stub (pendiente de implementar).
CI/CD y migraciones multi-tenant	Pendiente — el demo no usa multi-tenant ni Prisma (usa SQL plano); revisar en <code>apps/api</code> .
Estado de los 3 no-negociables (backups, credenciales, pipeline)	Pendiente — no aplica al demo standalone; verificar en el entorno de producción.
¿Demo sobre backend real o mock?	Respondido — standalone con Postgres + BFF propios.

7. Próximo paso recomendado

1. **Reconciliar tracks:** decidir formalmente que este frontend demo es el de producción y conectarlo al backend real (implementar `http` detrás de `client.ts`, pantalla por pantalla).
2. **Cablear lo financiero:** motor de costos + margen real + facturación/cobranza con datos reales.
3. **Construir la cadena comercial-fiscal de SET:** propuestas, divisor, presupuestos, aprobación granular, ODC, RFC, testigos, pruebas de color, reportes.
4. **Endurecer producción:** multi-tenant + RLS, backups, CI/CD con migraciones, credenciales encriptadas.

En una frase: el demo está ~85% como producto y convenció en los flujos; producción está ~20–25%. Lo que el análisis subestima es que el demo ya tiene auth real, BD y BFF; lo que acierta es que el núcleo comercial-fiscal y las integraciones reales son el gran pendiente.